Abstract Window Toolkit

Study Notes for OAVS Computer Teacher Competitive Examination

Java AWT • Concise & Exam-Focused

Topics Covered (in order):

1. Components and Graphics
2. Containers, Frames and Panels
3. Layout Managers (Border, Flow, Grid, Card)
4. AWT all Components
5. Event Delegation Model
6. Event Source and Handlers
7. Event Categories, Listeners, Adapters
8. Anonymous Classes
9. Applets (Life Cycle, Context, Inter-applet Communication)
10. Java Utility Packages, Classes, Interfaces
11. Streams and File IO

AWT Overview

Definition

Java AWT (Abstract Window Toolkit) is an API to develop Graphical User Interface (GUI) or window-based applications in Java, provided by the `java.awt` package. It is Java's original, platform-dependent toolkit for building desktop applications.

Key Characteristics

- AWT stands for **Abstract Window Toolkit** (also written "Abstract Windowing Toolkit" in some textbooks).
- AWT is used to develop **Desktop Applications**.
- AWT components are **platform-dependent** — i.e. components are displayed according to the look of the underlying operating system.
- AWT components are **heavyweight** — each component relies on a native OS peer for rendering.
- The `java.awt` package provides the classes and interfaces for the AWT API.

Peer-Based (Heavyweight) Architecture

Every AWT component is backed by a corresponding **native peer** object created by the host operating system's own GUI toolkit (e.g., Windows controls, X11/Motif widgets on Linux, Mac Toolbox on macOS). The AWT class itself is only a thin Java-side wrapper that delegates actual rendering and input handling to this native peer — this is why AWT components are called "heavyweight", and why their appearance changes from one OS to another.

Key AWT Sub-Packages

Package	Contains
java.awt	Core classes — Component, Container, Frame, Panel, Graphics, layout managers, basic controls
java.awt.event	Event classes and listener interfaces (ActionEvent, MouseListener, etc.)
java.awt.image	Classes for image creation, processing and manipulation
java.applet	Applet class and related applet-support classes

Exam Points

- AWT was introduced with the very first version of Java (JDK 1.0).
- "Abstract" in AWT refers to the abstraction layer over platform-specific native GUI peers.
- AWT components are heavyweight and platform-dependent — contrast with Swing, which is mostly lightweight and gives a consistent cross-platform look (Swing itself is outside this syllabus section).

1. Components and Graphics

1.1 Component Class

Definition

`java.awt.Component` is the **abstract superclass of all AWT GUI elements** (buttons, labels, checkboxes, containers, etc.). It is an object with a graphical representation that can be displayed on screen and can interact with the user through events.

Core Concept

- Component is **abstract** — it cannot be instantiated directly; subclasses like `Button` and `Label` are used.
- It defines the common properties of every GUI element (position, size, colours, font, visibility, enabled state, focus), the **painting mechanism** and **event registration** methods.
- A Component is placed on screen by adding it to a **Container**, which is itself a Component.

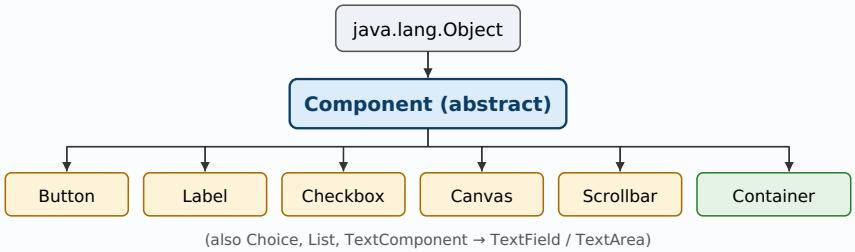


Fig 1.1: Component as the root of all AWT GUI elements

Component — Common Methods

Method	Purpose
setSize(w,h) / setBounds(x,y,w,h)	Sets size / position + size of the component
setVisible(boolean)	Shows or hides the component
setEnabled(boolean)	Enables / disables user interaction
setBackground(Color) / setForeground(Color)	Sets background / text colour
setFont(Font)	Sets the font used for text
getSize() / getLocation() / getBounds()	Returns current Dimension / Point / Rectangle
requestFocus()	Requests keyboard focus for the component
addXxxListener() / removeXxxListener()	Registers / removes an event listener

Painting Methods: paint() vs update() vs repaint()

Method	Role
paint(Graphics g)	Contains the actual drawing code; called by AWT to render the component
repaint()	Requests AWT to schedule a redraw; internally calls update(), which by default clears the background then calls paint()
update(Graphics g)	Clears the component and calls paint(); can be overridden to reduce flicker

Example: Working with Component Properties

```
import java.awt.*;

public class ComponentDemo {
    public static void main(String[] args) {
        Button b = new Button("OK");       // Button is a Component
        b.setBounds(10, 20, 80, 30);       // x, y, width, height
        b.setBackground(Color.YELLOW);

        System.out.println(b.getLabel());
        System.out.println(b.getBounds());
    }
}
```

Output:

```
OK
java.awt.Rectangle[x=10,y=20,width=80,height=30]
```

The button is never added to a window, so only these two lines are printed.

Canvas Class

Definition

`java.awt.Canvas` is a blank rectangular Component meant for custom drawing: a subclass overrides `paint()` to draw graphics, games or animations, instead of drawing directly on a Frame.

Exam Points

- Component is the root of the AWT class hierarchy for GUI elements (`Object → Component`).
- Component is abstract; `Container` is a subclass of Component, so a Container can itself be added to another Container.
- Component supplies `paint()`, `update()` and `repaint()`; `repaint()` calls `update()`, which calls `paint()`.

Summary: Component is the abstract base of every AWT GUI element. It provides position, size, colour, font, visibility, focus, painting and event-registration features that all controls and containers inherit.

1.2 Graphics Class

Definition

`java.awt.Graphics` is an abstract class that provides methods to draw shapes, text and images onto a component. An object of Graphics is passed automatically to the `paint(Graphics g)` method by the AWT system.

Important Drawing Methods

Method	Purpose
`drawString(String s, int x, int y)`	Draws text at position (x, y)
`drawLine(x1, y1, x2, y2)`	Draws a line between two points
`drawRect / fillRect(x, y, w, h)`	Draws / fills a rectangle
`drawOval / fillOval(x, y, w, h)`	Draws / fills an oval inside bounding box
`drawArc / fillArc(...)`	Draws / fills an arc
`setColor(Color c)`	Sets the current drawing color
`setFont(Font f)`	Sets the current font for text drawing

Example: Simple Drawing using paint()

```
import java.awt.*;

public class GraphicsDemo extends Frame {
    public void paint(Graphics g) {
        g.setColor(Color.BLUE);
        g.drawString("Hello AWT!", 50, 50);
        g.drawRect(30, 70, 100, 40);
        g.fillOval(150, 70, 60, 40);
    }
    public static void main(String[] args) {
        GraphicsDemo f = new GraphicsDemo();
        f.setSize(300, 200);
        f.setVisible(true);
    }
}
```

Output:



Output window: blue text "Hello AWT!", a rectangle outline and a filled oval (300 × 200 frame)

More Graphics Methods

Method	Purpose
`drawPolygon(int[] x, int[] y, int n)`	Draws a closed polygon from arrays of x/y points
`fillPolygon(...)`	Draws a filled polygon
`draw3DRect / fill3DRect(...)`	Draws a rectangle with a 3-D raised/lowered effect
`drawRoundRect / fillRoundRect(...)`	Rectangle with rounded corners
`clipRect(x, y, w, h)`	Restricts drawing to a rectangular clipping region
`drawImage(Image img, x, y, this)`	Draws an image on the component
`getColor() / getFont()`	Returns the current drawing color/font
`dispose()`	Releases the Graphics context's resources when no longer needed

Coordinate System

The origin (0,0) of a component's Graphics context is at the **top-left corner**. The x-axis increases rightward, and the y-axis increases **downward** (opposite of standard mathematics) — a key exam fact.

Color and Font Classes

Color

`java.awt.Color` represents RGB colors. Predefined constants: `Color.RED`, `Color.BLUE`, `Color.GREEN`, `Color.BLACK`, `Color.WHITE`, etc. Custom colors: `new Color(int r, int g, int b)` (each 0-255).

Font

`java.awt.Font` represents a text font. Constructor: `new Font(String name, int style, int size)` where style is `Font.PLAIN`, `Font.BOLD`, `Font.ITALIC`, or a combination (`Font.BOLD+Font.ITALIC`).

Example: Setting Color and Font

```
g.setColor(new Color(255, 0, 0));
g.setFont(new Font("Arial", Font.BOLD, 18));
g.drawString("Styled Text", 40, 60);
```

Output:



Red bold text "Styled Text" drawn at (40, 60)

Exam Points

- `paint()` is called automatically by AWT whenever the component needs to be redrawn.
- Component and Graphics both belong to `java.awt` package.
- Y-axis increases downward in AWT's coordinate system — opposite of standard graph convention.
- Graphics is abstract; the actual object is a native-peer-specific subclass supplied by AWT at runtime — programmers never instantiate it directly.

Summary: Graphics provides the drawing API (lines, shapes, text, images, color, font) used inside `paint()` to render custom graphics, with origin at top-left and y increasing downward.

2. Containers, Frames and Panels

2.1 Container

Definition

`java.awt.Container` is a subclass of Component that can hold other components/containers inside it. It supports adding components via `add()` and arranging them using a layout manager.

- Every Container has a default layout manager (e.g., Frame → BorderLayout, Panel → FlowLayout).
- Important methods: `add(Component c)`, `setLayout(LayoutManager lm)`, `remove(Component c)`.

Container — Important Methods

Method	Purpose
add(Component c)	Adds a component to the container
add(Component c, Object constraints)	Adds with layout-specific constraints (e.g., BorderLayout.NORTH)
remove(Component c) / removeAll()	Removes one / all components
getComponentCount()	Returns number of components in the container
getComponent(int i)	Returns component at given index
setLayout(LayoutManager lm)	Sets/changes the layout manager
validate() / doLayout()	Re-arranges components after changes

2.2 Frame

Definition

`java.awt.Frame` is a top-level window with a title bar, border, and (optionally) a menu bar. It is a subclass of `Window`, which is a subclass of `Container`.

- Default layout of Frame = **BorderLayout**.
- Created either by instantiating Frame directly or by extending the Frame class.
- Must call `setSize()` and `setVisible(true)` to display it.

Program: Creating a Frame

```
import java.awt.*;
public class MyFrame extends Frame {
    MyFrame() {
        setTitle("My First Frame");
        setSize(300, 200);
        setVisible(true);
    }
    public static void main(String[] args) { new MyFrame(); }
}
```

Output:



An empty 300 × 200 frame titled "My First Frame"

Frame — Constructors and Common Methods

Member	Description
Frame()	Creates an invisible frame with no title
Frame(String title)	Creates an invisible frame with the given title
setTitle(String)	Sets the frame's title bar text
setSize(int width, int height)	Sets the width and height of the frame (in pixels)
setResizable(boolean)	Allows/prevents user from resizing the frame
setIconImage(Image)	Sets the frame's title-bar icon
setMenuBar(MenuBar mb)	Attaches a menu bar to the frame
dispose()	Destroys the frame and releases its resources

Program: Frame with Close Handling

```
import java.awt.*;
import java.awt.event.*;
public class FrameDemo extends Frame {
    FrameDemo() {
        setTitle("Demo Frame");
        setSize(300, 200);
        setResizable(false);
        addWindowListener(new WindowAdapter() {
            public void windowClosing(WindowEvent e) { System.exit(0); }
        });
        setVisible(true);
    }
    public static void main(String[] args) { new FrameDemo(); }
}
```

Output:



A fixed-size 300 × 200 frame titled "Demo Frame" (not resizable)

Behaviour: `Clicking the close (X) button calls System.exit(0) and ends the program.`

2.3 Panel

Definition

`java.awt.Panel` is a generic, invisible container used to group components. Unlike Frame, a Panel has no title bar/border and cannot exist independently — it must be added to another container (like a Frame).

- Default layout of Panel = **FlowLayout**.
- Useful to organize sub-groups of components within a Frame (e.g., one panel for buttons, another for text fields).

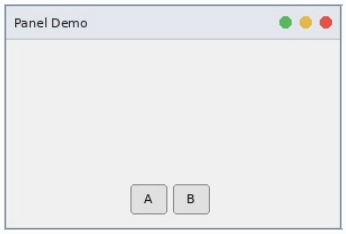
Panel — Constructors

Constructor	Description
Panel()	Creates a panel with default FlowLayout
Panel(LayoutManager lm)	Creates a panel with the specified layout manager

Example: Adding a Panel to a Frame

```
Frame f = new Frame("Panel Demo");
Panel p = new Panel();               // default FlowLayout
p.add(new Button("A"));
p.add(new Button("B"));
f.add(p, BorderLayout.SOUTH);        // panel placed inside frame's BorderLayout
f.setSize(300, 200);
f.setVisible(true);
```

Output:



Panel with buttons A and B (FlowLayout, centred) placed in the SOUTH region

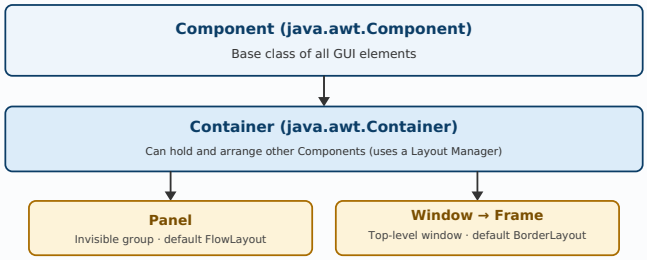


Fig 2.1: Component → Container → Panel / Frame relationship

Feature	Frame	Panel
Nature	Top-level window (independent)	Sub-container (cannot exist alone)
Title bar / border	Yes	No
Default layout	BorderLayout	FlowLayout
Parent class	Window	Container
Usage	Main application window	Grouping components inside a Frame

Window Class

Definition

`java.awt.Window` is a top-level window with **no border and no title bar** — the parent class of Frame and Dialog. It is rarely used directly since it lacks decorations.

Insets and ScrollPane

- **Insets** (`java.awt.Insets`) represent the border margins (top, left, bottom, right) of a container — obtained via `getInsets()`, used to leave space around the container's border for accurate component placement.
- **ScrollPane** is a specialised container that provides automatic horizontal/vertical scrolling for a single child component that is larger than the visible area.

Exam Points

- Hierarchy: `Component → Container → Window → Frame` and `Component → Container → Panel`.
- Frame's default layout = BorderLayout; Panel's default layout = FlowLayout — a frequently asked comparison.
- A Panel added inside a Frame can have its own independent layout manager.
- Window has no border/title bar; Frame and Dialog are its subclasses that add decoration.
- Closing a Frame does not terminate the JVM automatically — the programmer must handle `windowClosing()` and call `System.exit(0)` (or use `setDefaultCloseOperation` in Swing, not applicable to plain AWT).

Summary: Container groups components; Window is a bare top-level container; Frame (BorderLayout) is a decorated independent window built on Window; Panel (FlowLayout) is a nested, borderless grouping container that must live inside another container; Insets/ScrollPane assist with spacing and scrolling.

3. Layout Managers

Definition

A Layout Manager (interface `java.awt.LayoutManager`) automatically arranges components within a container, deciding size and position — avoiding manual pixel placement and making GUIs resize-friendly.

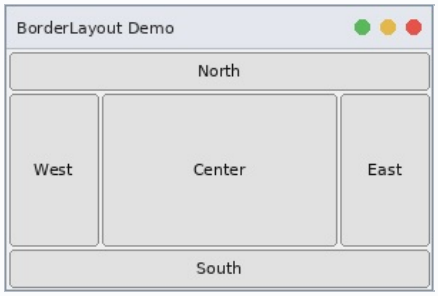
3.1 Border Layout

- Divides the container into **5 regions**: `North, South, East, West, Center`.
- Each region holds only **one** component; Center region takes up remaining space.
- Default layout of **Frame, Window and Dialog** — used when no layout is set; add() with no region goes to CENTER.

Example: BorderLayout

```
setLayout(new BorderLayout());
add(new Button("North"), BorderLayout.NORTH);
add(new Button("South"), BorderLayout.SOUTH);
add(new Button("East"),  BorderLayout.EAST);
add(new Button("West"),  BorderLayout.WEST);
add(new Button("Center"),BorderLayout.CENTER);
```

Output:



Frame (300 × 200) using BorderLayout

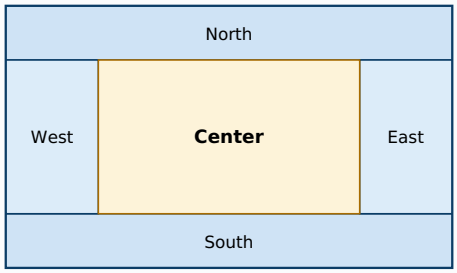


Fig 3.1: BorderLayout — 5 regions

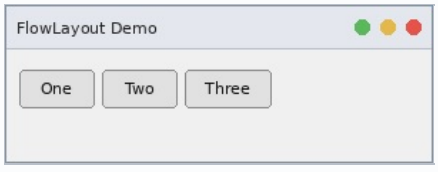
3.2 Flow Layout

- Arranges components **left to right**, wrapping to next line when the row is full (like words in a paragraph).
- Default layout of **Panel and Applet**.
- Alignment can be LEFT, RIGHT or CENTER (default = CENTER).

Example: FlowLayout

```
setLayout(new FlowLayout(FlowLayout.LEFT, 10, 10));
add(new Button("One"));
add(new Button("Two"));
add(new Button("Three"));
```

Output:



Buttons One, Two, Three placed left to right with FlowLayout.LEFT

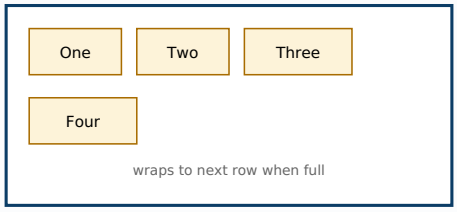


Fig 3.2: FlowLayout — left-to-right, wraps on overflow

3.3 Grid Layout

- Arranges components in a **rectangular grid** of equal-sized rows & columns.
- Constructor: `new GridLayout(rows, cols)` or `new GridLayout(rows, cols, hgap, vgap)`.
- All cells have the **same size**; components fill left-to-right, top-to-bottom.

Example: GridLayout

```
setLayout(new GridLayout(2, 3));
for (int i = 1; i <= 6; i++) add(new Button("B" + i));
```

Output:



Six equal-sized cells in a 2 × 3 grid

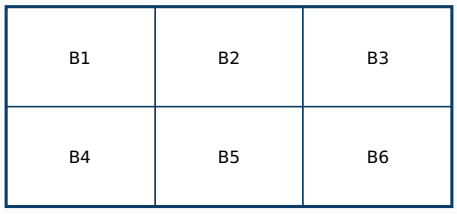


Fig 3.3: GridLayout(2, 3) — equal-sized cells

3.4 Card Layout

- Treats each component as a **"card"** in a stack — only **one card is visible** at a time (like a deck of cards / tabbed view).
- Useful for wizard-style screens or switching between multiple views.
- Navigation methods: `first()`, `next()`, `previous()`, `last()`, `show(Container, name)`.

CardLayout Method	Effect
first(Container c)	Shows the first card
next(Container c)	Shows the next card (wraps to first after last)
previous(Container c)	Shows the previous card
last(Container c)	Shows the last card
show(Container c, String name)	Shows the card registered with the given name

Example: CardLayout Methods

```
CardLayout cl = new CardLayout();
Panel p = new Panel(cl);
p.add(new Button("Card1"), "1");
p.add(new Button("Card2"), "2");
cl.show(p, "2");   // shows Card2
cl.next(p);        // moves to next card
```

Output:



Only one card is visible at a time

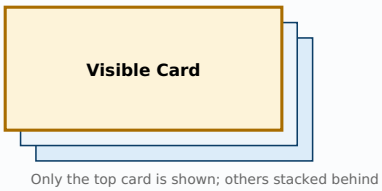


Fig 3.4: CardLayout — stacked cards, one visible

Constructors at a Glance

Layout	Constructors
BorderLayout	BorderLayout() | BorderLayout(int hgap, int vgap)
FlowLayout	FlowLayout() | FlowLayout(int align) | FlowLayout(int align, int hgap, int vgap)
GridLayout	GridLayout(int rows, int cols) | GridLayout(int rows, int cols, int hgap, int vgap)
CardLayout	CardLayout() | CardLayout(int hgap, int vgap)

LayoutManager Interface

> **Definition**
> All layout managers implement `java.awt.LayoutManager` (BorderLayout/GridLayout/CardLayout also implement the extended `LayoutManager2`). Key methods every layout manager provides internally:

Method	Purpose
addLayoutComponent(String name, Component comp)	Called when a component is added to the container (with a name/constraint)
removeLayoutComponent(Component comp)	Called when a component is removed from the container
layoutContainer(Container parent)	Positions and sizes all the components inside the container
preferredLayoutSize(Container parent)	Returns the preferred (ideal) size needed for the container's components
minimumLayoutSize(Container parent)	Returns the minimum size needed for the container's components

Null Layout (Absolute Positioning)

Calling `setLayout(null)` disables automatic layout management, requiring the programmer to manually place each component using `setBounds(x, y, width, height)`. This gives pixel-precise control but loses automatic resizing/platform independence — an important contrast point in exams.

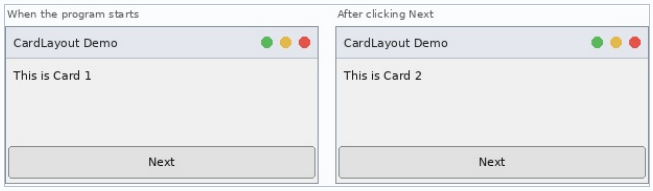
Complete CardLayout Example

```
Panel cards = new Panel();
CardLayout cl = new CardLayout();
cards.setLayout(cl);
cards.add(new Label("This is Card 1"), "card1");
cards.add(new Label("This is Card 2"), "card2");

Button nextBtn = new Button("Next");
nextBtn.addActionListener(e -> cl.next(cards));

Frame f = new Frame("CardLayout Demo");
f.setLayout(new BorderLayout());
f.add(cards, BorderLayout.CENTER);
f.add(nextBtn, BorderLayout.SOUTH);
f.setSize(300, 150);
f.setVisible(true);
```

Output:



Card 1 is shown first; each click on Next shows the next card

Comparison of Layout Managers

Layout	Arrangement	Default For	Key Point
BorderLayout	5 fixed regions (N,S,E,W,Center)	Frame/Window/Dialog	1 component per region
FlowLayout	Left-to-right, wraps	Panel/Applet	Natural size of components retained
GridLayout	Equal rows × columns	-	All cells equal size
CardLayout	Stack, one visible	-	Used for switching screens

Exam Points

- Frame default = BorderLayout; Panel/Applet default = FlowLayout (very commonly asked).
- GridLayout ignores preferred size of components — forces equal cell sizes.
- CardLayout is the only layout manager designed for showing one component at a time.

Summary: BorderLayout (5 regions), FlowLayout (sequential wrap), GridLayout (equal grid), CardLayout (stacked cards) — each solves a different arrangement need.

4. AWT all Components

Definition

AWT provides a rich set of ready-made GUI controls (all subclasses of `Component`) for building interfaces — buttons, labels, text fields, checkboxes, lists, menus, scrollbars, etc.

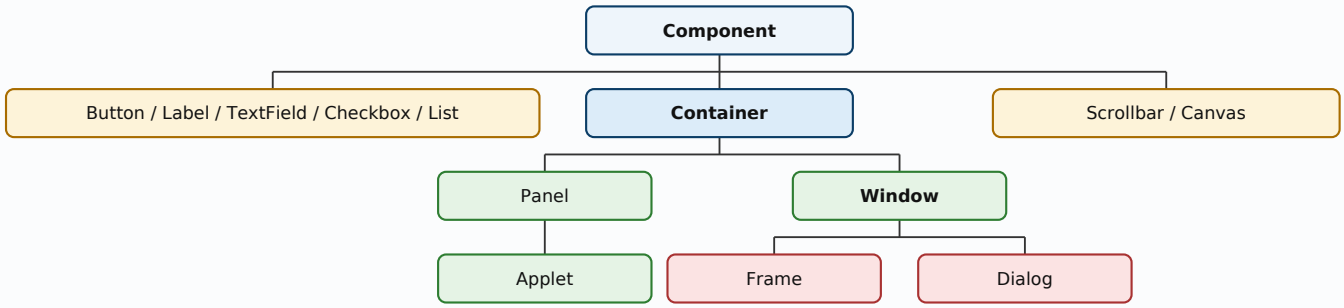


Fig 4.1: AWT Component Hierarchy (simplified)

Commonly Used AWT Components

Component	Purpose
Label	Displays non-editable text
Button	Triggers an action when clicked
TextField	Single-line editable text input
TextArea	Multi-line editable text input
Checkbox	On/off toggle; group of Checkboxes = radio buttons (CheckboxGroup)
Choice	Drop-down list (single selection, pop-up style)
List	Scrollable list, supports single/multiple selection
Scrollbar	Horizontal/vertical slider for scrolling a value/range
Canvas	Blank rectangular area for custom drawing
MenuBar / Menu / MenuItem	Creates menu system attached to a Frame
Dialog	Pop-up window for user interaction (modal/non-modal)
FileDialog	Native file selection dialog
ScrollPane	Container providing automatic scrolling for one child

Label

Purpose: Displays non-editable text.

```
Label lbl = new Label("Name:", Label.LEFT);  // alignment: LEFT/RIGHT/CENTER
add(lbl);
```

Output:



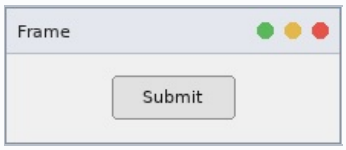
Label added to a Frame (BorderLayout, CENTER) — text is left-aligned

Button

Purpose: Triggers an action when the user clicks it.

```
Button btn = new Button("Submit");
btn.addActionListener(e -> System.out.println("Submitted"));
add(btn);
```

Output:



Button added to a frame (FlowLayout)

Output:

```
Submitted
```

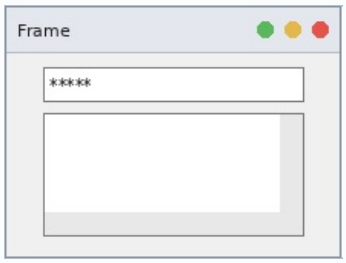
Printed once for every click on the button.

TextField and TextArea

Purpose: Accept editable text input (TextField: one line, TextArea: many lines).

```
TextField tf = new TextField(20);           // 20 columns wide
TextArea ta = new TextArea(5, 30);           // 5 rows, 30 columns
tf.setEchoChar('*');                         // password-style masking
```

Output:



*Text field (echo character * after typing 5 characters) above a text area*

Method	Purpose
getText() / setText(String)	Reads / sets the current text
setEditable(boolean)	Makes the field read-only or editable
setEchoChar(char)	(TextField only) masks input, e.g. for passwords

Checkbox and CheckboxGroup

Purpose: On/off choice; a CheckboxGroup allows only one selection (radio buttons).

```
CheckboxGroup cg = new CheckboxGroup();
Checkbox male = new Checkbox("Male", cg, true);
Checkbox female = new Checkbox("Female", cg, false);
add(male); add(female);   // mutually exclusive - acts like radio buttons
```

Output:



CheckboxGroup: only one option can be selected

Choice

Purpose: Drop-down list from which one item is selected.

```
Choice c = new Choice();
c.add("Delhi"); c.add("Mumbai"); c.add("Chennai");
add(c);
System.out.println(c.getSelectedItem());
```

Output:



Choice (drop-down) showing the default first item

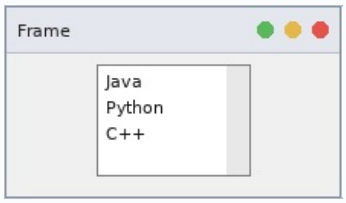
Output:

```
Delhi
```

List **Purpose:** Scrollable list; allows single or multiple selection.

```
List list = new List(4, true);   // 4 visible rows, multi-select = true
list.add("Java"); list.add("Python"); list.add("C++");
add(list);
String[] chosen = list.getSelectedItems();
```

Output:

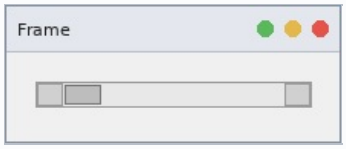


List with 4 visible rows (multiple selection allowed)

Scrollbar **Purpose:** Scrolls through or selects a value within a range.

```
Scrollbar sb = new Scrollbar(Scrollbar.HORIZONTAL, 0, 10, 0, 100);
// (orientation, initial value, visible amount, min, max)
add(sb);
```

Output:



Horizontal Scrollbar, slider at its minimum (0)

Menu System **Purpose:** Menu bar with drop-down items attached to a Frame.

```
MenuBar mb = new MenuBar();
Menu fileMenu = new Menu("File");
MenuItem openItem = new MenuItem("Open");
MenuItem exitItem = new MenuItem("Exit");
fileMenu.add(openItem);
fileMenu.add(exitItem);
mb.add(fileMenu);
frame.setMenuBar(mb);
exitItem.addActionListener(e -> System.exit(0));
```

Output:



"File" menu opened on the frame's menu bar

Dialog and FileDialog **Purpose:** Pop-up windows: Dialog for user input, FileDialog to choose a file.

- **Dialog** can be **modal** (blocks input to other windows until closed) or **non-modal**. Constructor: `new Dialog(Frame parent, String title, boolean modal)`.
- **FileDialog** shows the native Open/Save file picker: `new FileDialog(Frame parent, String title, int mode)` where mode is `FileDialog.LOAD` or `FileDialog.SAVE`.

> **Modal vs Non-modal Dialog**
> - **Modal** (`true`): blocks the other windows until the dialog is closed.
> - **Non-modal** (`false`): other windows stay usable while it is open.
> - Set by the 3rd constructor argument: `new Dialog(frame, "Title", true)`

Exam Points

- All AWT components are heavyweight (rely on native peers) — contrast with Swing's lightweight components (often asked as a concept, not to be confused as an AWT topic itself).
- CheckboxGroup is used to make Checkboxes behave like mutually-exclusive radio buttons.
- Choice shows one item at a time with a drop-down; List can show multiple items and allow multi-select.
- TextField's `setEchoChar()` is commonly used for password inputs (AWT has no dedicated PasswordField).
- A Dialog is normally given an owner (parent) Frame; unlike Frame, it is not the main window.

Summary: AWT offers a complete set of basic UI controls (Label, Button, TextField, TextArea, Checkbox/CheckboxGroup, Choice, List, Scrollbar, Canvas, Menu/MenuItem, Dialog, FileDialog) — all derived from Component and placed inside Containers using layout managers.

5. Event Delegation Model

Definition

The Event Delegation Model (EDM) is the standard mechanism (since Java 1.1) used by AWT/Swing to handle events, where a **source** component generates an event and **delegates** (passes) its handling to one or more registered **listener** objects, instead of the source handling the event itself.

Key Participants

- **Event Source** — the component that generates the event (e.g., Button).
- **Event Object** — encapsulates information about the event (e.g., ActionEvent).
- **Event Listener** — an object implementing a listener interface, registered with the source, whose callback method is invoked when the event occurs.

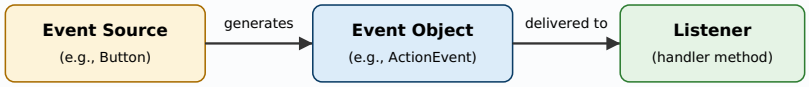


Fig 5.1: Event Delegation Model — Source → Event Object → Listener

Old (JDK 1.0) Event Model vs Event Delegation Model

Feature	Old Model (JDK 1.0)	Event Delegation Model (JDK 1.1+)
Approach	Inheritance-based — component had to be subclassed and `action()`/`handleEvent()` overridden	Delegation-based — separate listener object registered with source
Event routing	Every event passed up the containment hierarchy until handled	Event delivered directly to registered listener(s) only
Performance	Slow — all components tested for every event	Fast — no unnecessary checking
Flexibility	Low — required subclassing every component	High — any object implementing the listener interface can handle events

Steps in the Event Delegation Model

1. Create the event source component (e.g., Button).
2. Create a class implementing the relevant listener interface and define the handler method.
3. Register the listener object on the source using `addXxxListener()`.
4. When the event occurs, the source constructs an Event object and invokes the listener's handler method, passing that object.

Advantages of the Event Delegation Model

- Clear separation between GUI components (sources) and event-handling logic (listeners).
- Multiple listeners can be attached to one source, and one listener can serve multiple sources.
- Improves maintainability — event-handling code can live in any class, not just the component subclass.

Exam Points

- Introduced in JDK 1.1, replacing the older inefficient inheritance-based event model of JDK 1.0.
- Improves performance — only registered listeners receive events (no event checking in every component).
- Registration is done via `addXxxListener()` methods, e.g. `addActionListener()`; removal via corresponding `removeXxxListener()`.
- All event classes ultimately extend `java.util.EventObject`; AWT-specific events extend `java.awt.AWTEvent`.

Summary: Source generates an event, wraps details in an Event object, and delegates handling to a registered Listener's callback method — replacing the slower JDK 1.0 inheritance-based model.

6. Event Source and Handlers

6.1 Event Source

Definition

An **event source** is an object (a Component) capable of generating events. Examples: Button (ActionEvent), TextField (ActionEvent/TextEvent), Checkbox (ItemEvent), Frame (WindowEvent).

6.2 Event Handler

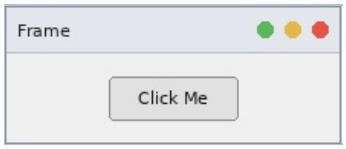
Definition

An **event handler** is the method inside a Listener implementation that contains the code executed in response to an event (e.g., `actionPerformed(ActionEvent e)`).

Registration Syntax

```
Button b = new Button("Click Me");
b.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        System.out.println("Button Clicked!");
    }
});
```

Output:



Button added to a frame

Output:

```
Button Clicked!
```

Printed each time the button is clicked.



Fig 6.1: Source → Listener registration → Handler method flow

Common Sources and the Events They Generate

Source Component	Typical Event Generated
Button	ActionEvent
TextField	ActionEvent (Enter key), TextEvent (text change)
Checkbox / Choice / List	ItemEvent
Scrollbar	AdjustmentEvent
Frame / Window	WindowEvent
Any Component	MouseEvent, KeyEvent, FocusEvent, ComponentEvent

Multiple Listeners on One Source

```
Button b = new Button("Click");
b.addActionListener(e -> System.out.println("Listener 1"));
b.addActionListener(e -> System.out.println("Listener 2"));
// both listeners' handler methods execute, in registration order
```

Output:



One button with two listeners

Output:

```
Listener 1
Listener 2
```

Both lines are printed on every click, in the order the listeners were registered.

One Listener Handling Multiple Sources

```
ActionListener commonHandler = e -> System.out.println("Source: " + e.getActionCommand());
Button b1 = new Button("Save");
Button b2 = new Button("Cancel");
b1.addActionListener(commonHandler);
b2.addActionListener(commonHandler);
// getActionCommand() identifies which button was pressed
```

Output:



Two buttons sharing one listener

Output:

```
Source: Save
Source: Cancel
```

Printed after clicking Save, then Cancel.

Exam Points

- A single source can have multiple listeners registered; a single listener can be registered with multiple sources.
- Handler method names/signatures are fixed by the listener interface (cannot be renamed).
- `getActionCommand()` on ActionEvent (default = button label) helps identify the source when one handler serves multiple buttons.
- `getSource()` (inherited from EventObject) returns the actual object that generated the event.

Summary: Source generates the event; the handler (a fixed callback method inside a registered listener) contains the response logic; sources and listeners can have a many-to-many relationship.

7. Event Categories, Listeners, Adapters

7.1 Event Categories (java.awt.event classes)

Every user action on a component creates an event object, and the class of that object decides its **category**. Each category in `java.awt.event` (e.g. `ActionEvent` for button clicks, `MouseEvent` for the mouse, `KeyEvent` for the keyboard) has a matching **listener interface** that handles it.

Event Class	Generated When
ActionEvent	Button clicked, menu item selected, Enter pressed in TextField
ItemEvent	Checkbox/Choice/List item state changes
MouseEvent	Mouse clicked, pressed, released, moved, dragged
KeyEvent	Key pressed, released, typed
WindowEvent	Window opened, closed, iconified, activated
FocusEvent	Component gains/loses focus
TextEvent	Text value changes in TextField/TextArea
AdjustmentEvent	Scrollbar value adjusted
ComponentEvent	Component resized, moved, shown, hidden
ContainerEvent	Component added/removed from container

7.2 Listener Interfaces

Definition

A **listener interface** declares one or more callback methods that a class must implement to receive and handle a specific category of event.

Listener Interface	Key Method(s)
ActionListener	actionPerformed(ActionEvent e)
ItemListener	itemStateChanged(ItemEvent e)
MouseListener	mouseClicked, mousePressed, mouseReleased, mouseEntered, mouseExited
MouseMotionListener	mouseDragged, mouseMoved
KeyListener	keyPressed, keyReleased, keyTyped
WindowListener	windowOpened, windowClosing, windowClosed, windowIconified, windowActivated ...(7 methods)
FocusListener	focusGained, focusLost
TextListener	textValueChanged(TextEvent e)
AdjustmentListener	adjustmentValueChanged(AdjustmentEvent e)

7.3 Adapter Classes

Definition

An **Adapter class** provides an empty (default) implementation of all methods of a listener interface that has more than one method. Programmers extend the adapter and override **only the methods they need**, avoiding forced implementation of every method.

Adapter Class	Implements Interface
WindowAdapter	WindowListener (7 methods)
MouseAdapter	MouseListener (5 methods)
MouseMotionAdapter	MouseMotionListener (2 methods)
KeyAdapter	KeyListener (3 methods)
FocusAdapter	FocusListener (2 methods)
ContainerAdapter	ContainerListener (2 methods)
ComponentAdapter	ComponentListener (4 methods)

Full Method List — WindowListener (via WindowAdapter)

Method	Called When
windowOpened(WindowEvent e)	Window is first opened
windowClosing(WindowEvent e)	User clicks the close (X) button
windowClosed(WindowEvent e)	Window has been closed (after dispose())
windowIconified(WindowEvent e)	Window is minimized
windowDeiconified(WindowEvent e)	Window is restored from minimized state
windowActivated(WindowEvent e)	Window becomes the active window
windowDeactivated(WindowEvent e)	Window loses active status

Full Method List — MouseListener / MouseMotionListener

Method	Called When
mouseClicked(MouseEvent e)	Mouse button pressed and released at same point
mousePressed(MouseEvent e)	Mouse button pressed down
mouseReleased(MouseEvent e)	Mouse button released
mouseEntered(MouseEvent e)	Mouse pointer enters the component
mouseExited(MouseEvent e)	Mouse pointer exits the component
mouseDragged(MouseEvent e) *(Motion)*	Mouse moved while a button is held
mouseMoved(MouseEvent e) *(Motion)*	Mouse moved with no button held

Example: Closing a Frame with WindowAdapter

```
frame.addWindowListener(new WindowAdapter() {
    public void windowClosing(WindowEvent e) {
        System.exit(0);   // override only this one method
    }
});
```

Output:



Clicking the close (X) button ends the program; nothing is printed

Extending an Adapter — Full Example

```
import java.awt.*;
import java.awt.event.*;

public class AdapterDemo extends Frame {
    AdapterDemo() {
        addWindowListener(new WindowAdapter() {
            public void windowClosing(WindowEvent e) {
                System.out.println("Closing window...");
                System.exit(0);
            }
        });
        setSize(250, 150);
        setVisible(true);
    }
    public static void main(String[] args) { new AdapterDemo(); }
}
```

Output:



Frame (250 × 150) with a WindowAdapter

Output:

```
Closing window...
```

Printed when the close (X) button is clicked; the program then exits.

Exam Points

- Adapter classes exist only for listener interfaces with **multiple** methods (e.g. no "ActionAdapter" since ActionListener has just one method, and no "ItemAdapter" since ItemListener has just one method).
- Using adapters reduces boilerplate code — very common exam MCQ.
- Adapter classes are abstract classes with empty method bodies, not interfaces.
- A class extending an Adapter can override only the methods it actually needs — unused methods keep the inherited empty body.
- Because Java has single inheritance, extending an Adapter class means that class cannot extend any other class.

Summary: Event categories map to specific Event classes; Listener interfaces define handler methods for each category; Adapter classes give default empty implementations to simplify coding when only some methods are needed.

8. Anonymous Classes

Definition

An **anonymous class** is a class without a name, declared and instantiated in a single expression — commonly used to implement listener interfaces or extend adapter classes inline, especially for event handling in AWT.

Example

```
Button b = new Button("OK");
b.addActionListener(new ActionListener() {   // anonymous class
    public void actionPerformed(ActionEvent e) {
        System.out.println("OK Pressed");
    }
});
```

Output:



Button added to a frame

Output:

```
OK Pressed
```

Printed each time the button is clicked.

Anonymous Class Extending an Adapter

```
frame.addMouseListener(new MouseAdapter() {   // extends a class anonymously
    public void mouseClicked(MouseEvent e) {
        System.out.println("Clicked at: " + e.getX() + "," + e.getY());
    }
});
```

Output:



Frame with a mouse listener

Output:

```
Clicked at: 57,102
```

Coordinates depend on where the mouse is clicked.

Two Forms of Anonymous Classes

Form	Example
Implementing an interface	`new ActionListener() { public void actionPerformed(ActionEvent e){...} }`
Extending a class (or adapter)	`new WindowAdapter() { public void windowClosing(WindowEvent e){...} }`

Anonymous Class vs Named (Inner) Class

Feature	Anonymous Class	Named Inner Class
Name	None	Has a name, reusable
Reusability	Used only once at declaration point	Can be instantiated multiple times
Constructors	Not allowed	Allowed
Extends/Implements	Exactly one class OR one interface	Can implement multiple interfaces
Typical Use	Quick, one-off event listeners	Reusable, more complex listener logic

Exam Points

- Anonymous classes can implement an interface or extend a class, but not both simultaneously.
- Cannot have explicit constructors (no name to define one) — but instance initializer blocks are allowed.
- Compiled with a name like `Outer$1.class` by the compiler.
- Commonly used with adapter classes to override only required methods concisely.
- Can access final (or effectively final) local variables of the enclosing method.
- Cannot define static members (except constant variables).

Summary: Anonymous classes let event listeners (or adapter overrides) be defined inline without creating a separate named class — widely used in AWT event handling for concise, one-off implementations.

9. Applets

Definition

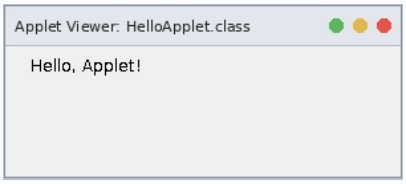
An **Applet** is a small Java program that runs inside a web browser or an applet viewer, embedded using the `<applet>` HTML tag (or via `appletviewer`). It extends `java.applet.Applet` (which extends Panel).

Program: A Simple Applet

```
import java.applet.*;
import java.awt.*;

public class HelloApplet extends Applet {
    public void paint(Graphics g) {
        g.drawString("Hello, Applet!", 20, 20);
    }
}
/* HTML: <applet code="HelloApplet.class" width="300" height="100"></applet> */
```

Output:



Applet Viewer window (300 × 100 applet) showing the text "Hello, Applet!"

9.1 Applet Life Cycle

The **applet life cycle** is the sequence of methods the browser calls on an applet, from the moment it is loaded until it is removed. Five methods control it: `init()`, `start()`, `paint()`, `stop()` and `destroy()`.

Method	Called When
init()	Once, when the applet is first loaded — used for initialization
start()	After init(), and every time the applet page is revisited — begins execution
paint(Graphics g)	Whenever the applet's display needs to be redrawn
stop()	When the browser leaves the applet's page — pauses execution
destroy()	Once, when the browser shuts down / applet is removed from memory — cleanup

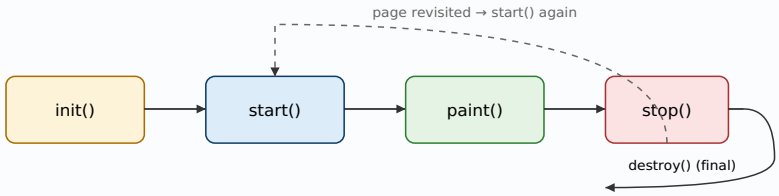


Fig 9.1: Applet Life Cycle — init → start → paint → (stop/start loop) → destroy

9.2 Applet Context

Definition

`AppletContext` is an interface that lets an applet obtain information about its environment (the browser) — such as other applets on the same page, and lets it request the browser to load a document or play audio.

- Obtained via `getAppletContext()`.

AppletContext — Key Methods

Method	Purpose
getApplet(String name)	Returns the applet with the given name on the same page (null if none)
getApplets()	Returns an Enumeration of all the applets on the same page
showDocument(URL)	Asks the browser to display the document at the given URL
showStatus(String)	Displays a message in the browser's status bar
getAudioClip(URL)	Returns an AudioClip object that can be played

9.3 Inter-applet Communication

Definition
Inter-applet communication allows multiple applets running on the **same web page** to interact with each other, using `AppletContext` to locate a named applet and invoke its public methods directly.

Example: Calling Another Applet's Method

```
// Applet B has name="appletB" in its <applet> tag; Applet A gets it and calls its method
AppletContext ctx = getAppletContext();
AppletB b = (AppletB) ctx.getApplet("appletB");
b.updateMessage("Hello from Applet A");
```

Output:



Applet B shows the message sent by Applet A (assuming updateMessage() displays its argument)

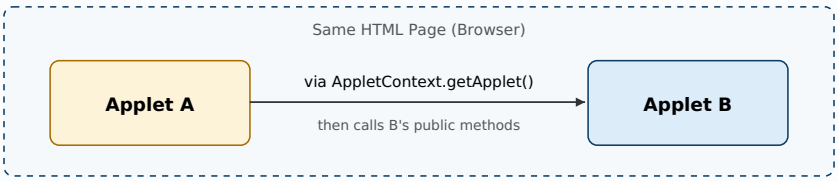


Fig 9.2: Inter-applet communication via AppletContext

9.4 The <applet> HTML Tag and Parameters

Attribute	Purpose
code	Name of the compiled .class file
width / height	Display dimensions of the applet
codebase	Directory where the .class file is located (optional)
archive	Names a JAR file containing the applet's classes

Definition
A **JAR file** (Java ARchive, `.jar`) is a single compressed file that bundles Java class files together with their resources, such as images, sounds and configuration files. It uses the same format as a ZIP file.

Custom parameters are passed via `<param>` tags and read inside the applet using `getParameter(String name)`:

Example: Passing Parameters to an Applet

```
<applet code="Welcome.class" width="300" height="100">
    <param name="username" value="Amit">
</applet>

// Inside the applet:
public void init() {
    String user = getParameter("username");
    System.out.println("Hello " + user);
}
```

Output:
```
Hello Amit
```

Applets can also supply self-description via two optional methods:

- `getAppletInfo()` — returns a String describing the applet (author, version).
- `getParameterInfo()` — returns a 2-D array describing each expected parameter.

9.5 Applet Security Restrictions ("Sandbox")

Definition
Since applets run inside a browser and are downloaded from possibly untrusted sources, they execute in a restricted **sandbox** that limits what they can do on the host machine.

- Cannot read/write files on the local (client) file system.
- Cannot run other local programs or load native libraries.
- Cannot generally make network connections to any host other than the one they were loaded from.
- Cannot read certain system properties that could reveal sensitive information.

9.6 Applet vs Application

Feature	Applet	Application
Execution	Runs inside browser/appletviewer	Runs standalone via JVM
Entry point	init()/start() (no main())	main() method
File system access	Restricted (sandboxed)	Full access (subject to OS permissions)
Life cycle	Managed by browser (5 methods)	Managed by programmer

Exam Points

- Applet class hierarchy: `Component → Container → Panel → Applet`.
- init() runs only once; start()/stop() can run multiple times; destroy() runs only once.
- Applets have no main() method; execution is controlled by the browser/appletviewer through life-cycle methods.
- Default layout of an Applet = FlowLayout (inherited from Panel).
- getParameter() always returns a String — must be manually converted (e.g. Integer.parseInt()) for other types.
- Modern browsers have dropped Java Applet/plugin support; appletviewer (part of older JDKs) is used for testing/exam purposes.

Summary: Applets run inside a browser with a fixed 5-method life cycle (init, start, paint, stop, destroy), receive parameters via the <param> tag, operate under strict sandbox security, and differ from standalone applications in execution model and file-system access; AppletContext exposes the browser environment and enables communication between applets on the same page.

10. Java Utility Packages, Classes, Interfaces

Definition

The `java.util` package provides utility classes for data structures, and `java.lang` provides fundamental classes automatically available to every program — together supporting collections, math operations, randomness, system interaction and string handling.



Fig 10.1: Utility classes grouped by package

10.1 HashTable

Definition

`java.util.Hashtable` stores data as **key-value pairs**, using a hash function for fast retrieval. It is synchronized (thread-safe) and does **not allow null** keys or values.

Method	Purpose	Example
put(key, value)	Inserts a key-value pair	`ht.put(1, "One")`
get(key)	Retrieves the value for a key	`ht.get(1) → "One"`
remove(key)	Deletes an entry	`ht.remove(1) → "One"`
containsKey(key) / containsValue(v)	Checks existence	`ht.containsKey(2) → true`
keys() / elements()	Returns an Enumeration of keys / values	`Enumeration<Integer> e = ht.keys();`
size()	Returns the number of entries	`ht.size() → 2`

Example: Hashtable

```
Hashtable<Integer, String> ht = new Hashtable<>();
ht.put(1, "One");
ht.put(2, "Two");
System.out.println(ht.get(1));
```

Output:

```
One
```

10.2 Vector

Definition

`java.util.Vector` is a growable/dynamic array that can hold objects of any type; it is synchronized and grows automatically (doubles in size) when it exceeds capacity.

Method	Purpose	Example
add(element) / addElement(element)	Appends an element	`v.add("Apple")`
get(index) / elementAt(index)	Retrieves element at index	`v.get(0) → "Apple"`
remove(index) / removeElement(obj)	Removes an element	`v.remove(0) → "Apple"`
size()	Number of elements currently stored	`v.size() → 2`
capacity()	Current allocated storage capacity	`new Vector<String>().capacity() → 10`
firstElement() / lastElement()	Returns first / last element	`v.lastElement() → "Ball"`

Example: Vector

```
Vector<String> v = new Vector<>();
v.add("Apple");
v.add("Ball");
System.out.println(v);
```

Output:
```
[Apple, Ball]
```

10.3 Math

Definition

`java.lang.Math` is a final class providing static methods/constants for mathematical operations (power, square root, trigonometry, rounding, absolute value, min/max).

Method	Purpose	Example
Math.PI / Math.E	Mathematical constants pi and e	`Math.PI → 3.141592653589793`
sqrt(x) / pow(x,y)	Square root / x raised to power y	`Math.sqrt(25) → 5.0` `Math.pow(2, 3) → 8.0`
abs(x)	Absolute value	`Math.abs(-7) → 7`
max(a,b) / min(a,b)	Larger / smaller of two values	`Math.max(4, 9) → 9`
round(x)	Rounds to nearest integer/long	`Math.round(4.6) → 5`
ceil(x) / floor(x)	Rounds up / down to nearest integer (as double)	`Math.ceil(4.2) → 5.0` `Math.floor(4.8) → 4.0`
random()	Returns a double between 0.0 (inclusive) and 1.0 (exclusive)	`Math.random() → e.g. 0.73`

Example: Math Methods

```
System.out.println(Math.sqrt(25));   // 5.0
System.out.println(Math.pow(2, 3));  // 8.0
System.out.println(Math.max(4, 9));  // 9
```

Output:
```
5.0
8.0
9
```

10.4 Random

Definition

`java.util.Random` generates pseudo-random numbers (int, double, boolean, long) using a seed-based algorithm.

Method	Purpose	Example
nextInt()	Random int across the full int range	`r.nextInt() → any int`
nextInt(int bound)	Random int in range 0 (inclusive) to bound (exclusive)	`r.nextInt(100) → 0 to 99`
nextDouble()	Random double between 0.0 and 1.0	`r.nextDouble() → e.g. 0.42`
nextBoolean()	Random true/false	`r.nextBoolean() → true or false`
setSeed(long)	Sets the seed — same seed produces the same sequence	`r.setSeed(5)`

Example: Generating a Random Number

```
Random r = new Random();
int n = r.nextInt(100);   // random number 0-99
```

Output:
```
42
```

Random value between 0 and 99; it changes on every run.

10.5 System

Definition

`java.lang.System` is a final class providing useful system-level fields/methods: standard I/O streams (`in`, `out`, `err`), `currentTimeMillis()`, `arraycopy()`, `exit()`, `gc()`.

Method	Purpose	Example
System.out / System.in / System.err	Standard output / input / error streams	`System.out.println("Hi")`
currentTimeMillis()	Current time in milliseconds since epoch (for timing code)	`long t = System.currentTimeMillis();`
arraycopy(src,sp,dest,dp,len)	Fast native array copy	`System.arraycopy(a, 0, b, 0, 3)`
exit(int status)	Terminates the JVM (0 = normal, non-zero = abnormal)	`System.exit(0)`
gc()	Suggests the JVM run garbage collection	`System.gc()`
getProperty(String)	Reads a system property (e.g. "os.name")	`System.getProperty("os.name")` → e.g. "Linux"

Example: Using System

```
System.out.println("Hello");
long t = System.currentTimeMillis();
```

Output:

```
Hello
```

10.6 String

Definition

`java.lang.String` represents an **immutable** sequence of characters — once created, its value cannot change; every modification creates a new String object.

Method	Purpose	Example
length()	Number of characters	`"Java".length()` → 4
charAt(i)	Character at index i	`"Java".charAt(1)` → 'a'
substring(begin, end)	Extracts a portion of the string	`"Hello".substring(1, 3)` → "el"
concat(str) / +	Concatenates strings (returns new String)	`"Hi".concat(" there")` → "Hi there"
equals() / equalsIgnoreCase()	Content comparison	`"abc".equals("ABC")` → false `"abc".equalsIgnoreCase("ABC")` → true
compareTo(str)	Lexicographic comparison	`"a".compareTo("b")` → -1
toUpperCase() / toLowerCase()	Case conversion (new String)	`"Java".toUpperCase()` → "JAVA"
trim()	Removes leading/trailing whitespace	`" hi ".trim()` → "hi"
indexOf(str) / replace(a,b)	Searching / replacing characters	`"Java".indexOf("v")` → 2 `"Java".replace('a', 'o')` → "Jovo"

Example: String Immutability

```
String s = "Hello";
s.concat(" World");
System.out.println(s);   // Hello (unchanged, immutable)
```

Output:

```
Hello
```

10.7 StringBuffer

Definition

`java.lang.StringBuffer` represents a **mutable** sequence of characters — content can be modified in place (append, insert, delete, reverse) without creating new objects. It is also synchronized/thread-safe.

Method	Purpose	Example
append(x)	Adds x to the end	`sb.append(" World")` → `"Hello World"`
insert(offset, x)	Inserts x at the given position	`new StringBuffer("Hlo").insert(1, "el")` → `"Hello"`
delete(start, end)	Removes characters in the range	`new StringBuffer("Hello").delete(1, 3)` → `"Hlo"`
reverse()	Reverses the character sequence	`new StringBuffer("abc").reverse()` → `"cba"`
replace(start, end, str)	Replaces characters in the range with str	`new StringBuffer("Hello").replace(0, 1, "J")` → `"Jello"`
capacity()	Returns current allocated buffer capacity	`new StringBuffer("Hello").capacity()` → 21

Example: StringBuffer

```
StringBuffer sb = new StringBuffer("Hello");
sb.append(" World");
System.out.println(sb);
```

Output:
```
Hello World
```

Feature	String	StringBuffer	Example
Mutability	Immutable	Mutable	`s.concat("!")` leaves s unchanged; `sb.append("!")` changes sb itself
Thread-safety	Immutable, inherently safe	Synchronized methods	One StringBuffer can be shared safely between threads
Performance (many edits)	Slower (new object each time)	Faster (in-place modification)	Building text in a loop: use `sb.append(x)`

Feature	Vector	Hashtable	Example
Stores	Ordered list of elements	Key-value pairs	`v.add("Apple")` / `ht.put(1, "One")`
Access	By index	By key	`v.get(0)` / `ht.get(1)`
Null allowed	Elements can include null	No null key/value	`v.add(null)` works; `ht.put(1, null)` throws NullPointerException
Synchronization	Synchronized	Synchronized	Both are thread-safe (legacy classes)

Exam Points

- Vector and Hashtable are legacy classes (pre-date the Collections Framework) but still synchronized/thread-safe — frequently contrasted with ArrayList/HashMap (not in this scope).
- String pool: String literals are stored in a special memory area called the String Constant Pool for reuse.
- Math class constructor is private — all members are static; cannot be instantiated.

Summary: java.util supplies Hashtable (key-value), Vector (dynamic array) and Random (randomness); java.lang supplies Math (calculations), System (system services), String (immutable text) and StringBuffer (mutable text).

11. Streams and File IO

Definition

The `java.io` package supports Input/Output using the concept of a **Stream** — an ordered sequence of data flowing from a source to a destination (file, keyboard, network, memory).

11.1 Files and Stream

- A **File** object (`java.io.File`) represents a file or directory **pathname** — it does not contain the file's data, only metadata (name, path, size, permissions).
- A **Stream** is the channel used to actually read/write the data of a file (or other source).



Fig 11.1: Stream data flow between source and destination

11.2 Stream Classes

- **Byte Streams** — handle raw binary data, 1 byte at a time; base classes `InputStream` / `OutputStream` (abstract).
- **Character Streams** — handle 16-bit Unicode characters (text), better for internationalized text; base classes `Reader` / `Writer` (abstract).

Category	Base Classes	Common Subclasses
Byte Stream (Input)	InputStream	FileInputStream, BufferedInputStream, DataInputStream, ObjectInputStream
Byte Stream (Output)	OutputStream	FileOutputStream, BufferedOutputStream, DataOutputStream, ObjectOutputStream, PrintStream
Character Stream (Input)	Reader	FileReader, BufferedReader, InputStreamReader
Character Stream (Output)	Writer	FileWriter, BufferedWriter, PrintWriter, OutputStreamWriter

Standard I/O Streams

Stream	Type	Purpose
System.in	InputStream	Standard input (keyboard), usually wrapped with InputStreamReader/BufferedReader
System.out	PrintStream	Standard output (console)
System.err	PrintStream	Standard error output

Example: Reading Keyboard Input

```
BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
String name = br.readLine();
System.out.println("Hello " + name);
```

Output:

```
Ravi
Hello Ravi
```

The first line (Ravi) is typed by the user.

DataInputStream / DataOutputStream

Definition

These allow reading/writing Java's **primitive data types** (int, double, boolean, etc.) in a portable binary format, rather than raw bytes or text.

Example: Writing and Reading Primitive Data

```
DataOutputStream dos = new DataOutputStream(new FileOutputStream("d.dat"));
dos.writeInt(100);
dos.writeDouble(3.14);
dos.close();

DataInputStream dis = new DataInputStream(new FileInputStream("d.dat"));
System.out.println(dis.readInt());     // 100
System.out.println(dis.readDouble());  // 3.14
```

Output:
```
100
3.14
```

RandomAccessFile

Definition

`java.io.RandomAccessFile` allows both reading and writing to a file at any position (not just sequentially), using a file pointer that can be moved with `seek(long pos)`.

Example: Random Access to a File

```
RandomAccessFile raf = new RandomAccessFile("data.txt", "rw");
raf.seek(10);              // move file pointer to byte 10
raf.writeInt(500);
System.out.println(raf.length());
raf.close();
```

Output:
```
14
```

The int 500 is written at byte 10, so the file length becomes 14 bytes.

11.3 Reader / Writer Classes

Definition

`Reader` and `Writer` are abstract classes forming the root of the character-stream hierarchy, used for reading/writing text data efficiently and portably (Unicode-aware), unlike byte streams.

Example: Reading a Text File

```
FileReader fr = new FileReader("data.txt");
BufferedReader br = new BufferedReader(fr);
String line = br.readLine();
System.out.println(line);
br.close();
```

Output:
```
Welcome to Java IO
```

Assuming the first line of data.txt is "Welcome to Java IO".

try-with-resources (Auto-close Streams)

Definition

Any class implementing `Closeable`/`AutoCloseable` (all stream classes do) can be declared inside a `try(...)` block, and Java automatically closes it at the end — avoiding explicit `close()` calls and resource leaks.

```
try (BufferedReader br = new BufferedReader(new FileReader("data.txt"))) {
    String line;
    while ((line = br.readLine()) != null) {
        System.out.println(line);
    }
} catch (IOException e) {
    System.out.println("Error: " + e.getMessage());
}
// br is automatically closed here, even if an exception occurs
```

Output:

```
Welcome to Java IO
Learning AWT
```

Assuming data.txt contains these two lines; the file is closed automatically.

11.4 File Class Tests and Utilities

Definition

`java.io.File` provides methods to **test** and manipulate file/directory properties without opening a stream.

Method	Purpose
exists()	Tests whether file/directory exists
canRead() / canWrite()	Tests read/write permission
isFile() / isDirectory()	Tests type of pathname
length()	Returns file size in bytes
delete()	Deletes the file/directory
mkdir()	Creates a new directory
list()	Lists files inside a directory

Example: File Tests

```
File f = new File("test.txt");
System.out.println(f.exists());
System.out.println(f.length());
```

Output:

```
true
19
```

Assuming test.txt exists and is 19 bytes long.

11.5 Serialization and De-serialization

Definition

Serialization is the process of converting an object's state into a byte stream (for storage in a file or transmission over network). **De-serialization** is the reverse process — reconstructing the object from the byte stream.

- Class must implement the marker interface `java.io.Serializable` (no methods to implement).
- Uses `ObjectOutputStream.writeObject()` to serialize and `ObjectInputStream.readObject()` to de-serialize.
- Fields marked `transient` are skipped during serialization.

```
class Student implements Serializable {
    int id; String name;
    Student(int id, String name){ this.id=id; this.name=name; }
}

// Serialize
ObjectOutputStream oos = new ObjectOutputStream(new FileOutputStream("s.dat"));
oos.writeObject(new Student(1, "Amit"));
oos.close();

// De-serialize
ObjectInputStream ois = new ObjectInputStream(new FileInputStream("s.dat"));
Student s = (Student) ois.readObject();
System.out.println(s.id + " " + s.name);
```

Output:

```
1 Amit
```

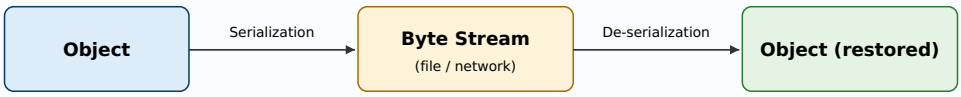


Fig 11.2: Serialization ↔ De-serialization process

Exam Points

- Byte streams: for binary data (images, audio); Character streams: for text data — classic exam distinction.
- Serializable is a marker interface (empty body) — just tags a class as serializable.
- static fields are never serialized (they belong to the class, not the object); transient fields are deliberately excluded.
- File class only handles file/directory metadata — actual reading/writing needs a Stream class.
- All stream I/O operations can throw `IOException` (checked exception) — must be handled with try-catch or declared with `throws`.
- RandomAccessFile supports mode "r" (read-only) or "rw" (read-write), and is the only stream class allowing non-sequential access via `seek()`.
- BufferedReader/BufferedWriter wrap other Reader/Writer streams to reduce the number of actual I/O operations, improving performance.

Summary: File represents a pathname; Streams (byte/character, via InputStream-OutputStream or Reader-Writer, plus Data/Buffered/RandomAccessFile variants) move actual data; File class methods test file properties; try-with-resources auto-closes streams; Serialization converts an object to a byte stream for storage/transmission, and de-serialization reconstructs it.

Quick Revision — AWT

Key Definitions

Component: Abstract superclass of all AWT GUI elements.

Container: Component that can hold other components; uses a layout manager.

Frame: Independent top-level window; default layout = BorderLayout.

Panel: Borderless sub-container; default layout = FlowLayout; cannot exist alone.

Event Delegation Model: Source generates event → wraps in Event object → delegates to registered Listener's handler method.

Adapter class: Provides default empty implementation of a multi-method listener interface, so only needed methods are overridden.

Anonymous class: Unnamed class defined and instantiated in one expression; used heavily for inline event listeners.

Applet life cycle: init() → start() → paint() → stop() → destroy() (init and destroy run only once).

Serialization: Object → byte stream (writeObject); De-serialization: byte stream → object (readObject); class must implement Serializable.

Must-Remember Differences

Compare	A	B
Frame vs Panel	Top-level, BorderLayout, has border	Sub-container, FlowLayout, no border
String vs StringBuffer	Immutable	Mutable
Vector vs Hashtable	Ordered list, index-based	Key-value pairs
Byte Stream vs Character Stream	InputStream/OutputStream, binary data	Reader/Writer, text/Unicode data
Listener vs Adapter	Interface — must implement ALL methods	Class — override only NEEDED methods

High-Yield Facts

- Default layout: Frame/Window/Dialog → BorderLayout; Panel/Applet → FlowLayout.
- GridLayout forces equal-sized cells; CardLayout shows only one component at a time.
- Event Delegation Model was introduced in JDK 1.1.
- Applet class hierarchy: Component → Container → Panel → Applet.
- Only init() and destroy() run exactly once in the applet life cycle; start()/stop() can repeat.
- Math and System classes have private constructors and only static members.
- transient and static fields are excluded from serialization.
- Serializable and Cloneable are **marker interfaces** (interfaces with no methods).